Data Processing and Modelling
Pandas, Scikit-Learn

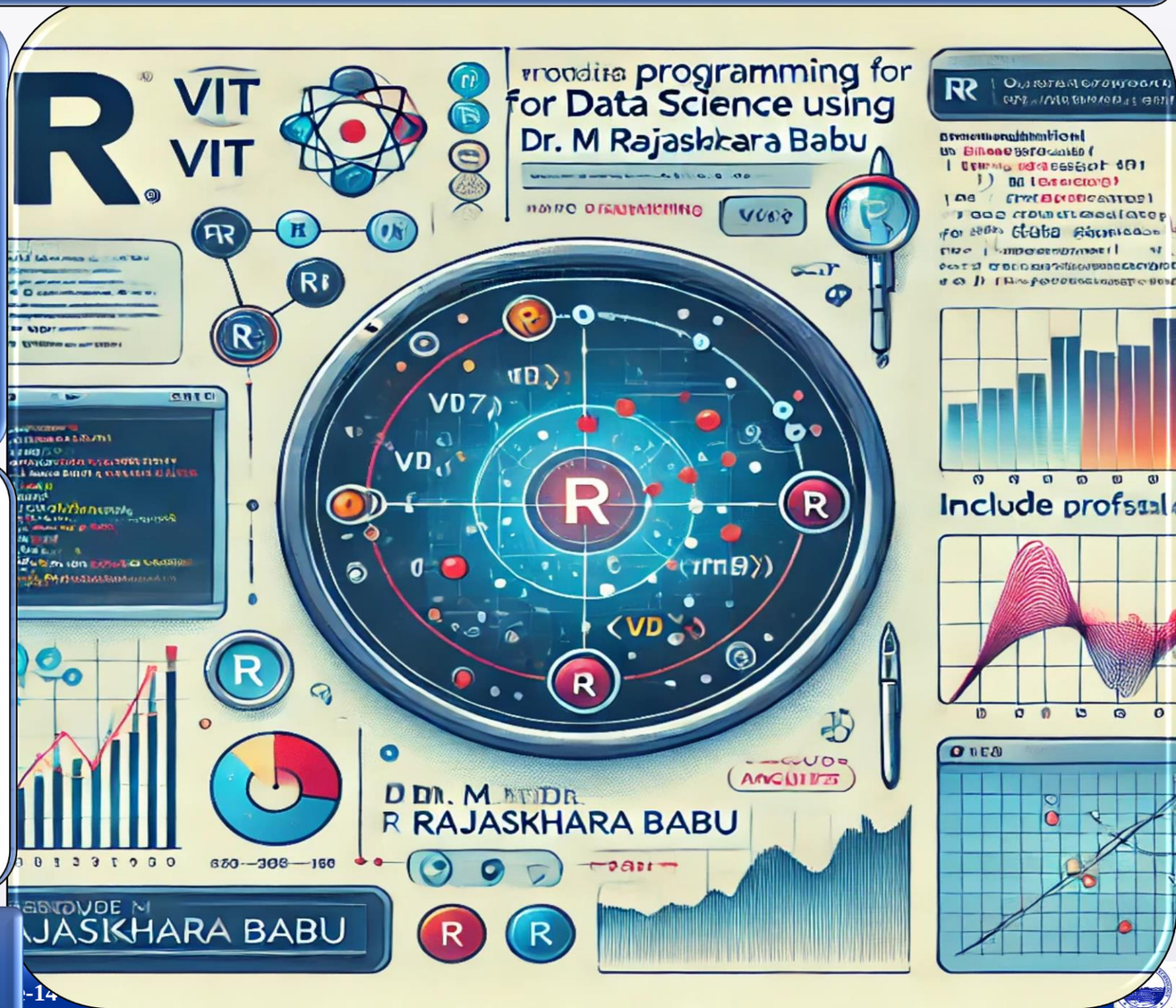
by

Dr M Rajasekhara Babu

School of

Computer Science and Engineering

https://www.rajababu.org



VELLORE INSTITUTE OF TECHNOLOGY
VIT
உழைப்பே உயர்வு தரும்

VIT®

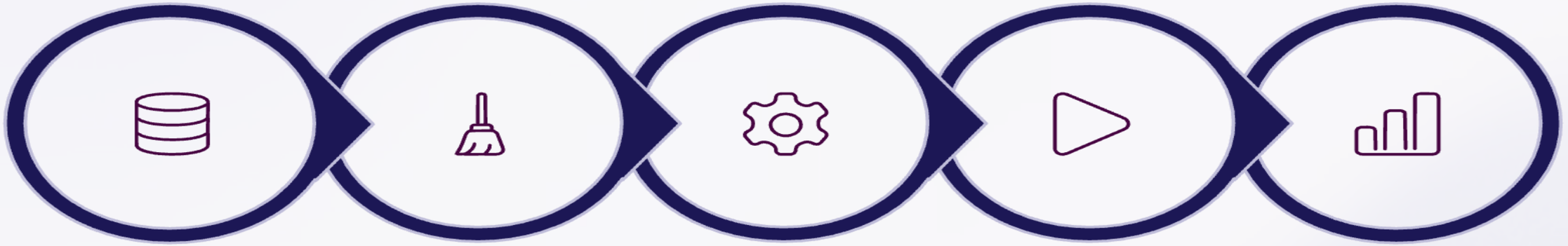
Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Vellore-632014, Tamil Nadu, India

Session Overview

The ML Pipeline



Tools We'll Use

Pandas
Data loading & transformation

Scikit-Learn
Classical ML algorithms

Keras
High-level deep learning API

TensorFlow
Deep learning framework

Introduction to Pandas for ML

Machine Learning algorithms require clean, structured, numerical data. Pandas is the go-to Python library for preparing your dataset before any model training begins.

Data Loading

Read CSV, Excel, JSON, and SQL sources with a single line of code.

Data Cleaning

Handle missing values, duplicates, and inconsistent formats.

Feature Selection & EDA

Select relevant columns and explore distributions before modeling.

Loading a Dataset with Pandas

Code

```
import pandas as pd

df =
pd.read_csv("students.csv")
print(df.head())
```

Sample Output

```
   Age  Score  Placement
0   20     85        Yes
1   21     90        Yes
2   22     65         No
```

Supported File Formats

- CSV
- Excel
- JSON
- SQL

Pandas provides a unified interface to load data from virtually any structured source into a DataFrame for immediate analysis.

Handling Missing Values & Feature Selection

Detecting & Fixing Missing Data

```
df.isnull().sum()       # Detect
df.fillna(df.mean()) # Fill with mean
df.dropna()             # Remove rows
```

Selecting Features & Target

```
X = df[['Age', 'Score']]  # Features
y = df['Placement']         # Target
```

- X holds the independent variables (inputs),
- while y holds the target variable (output) the model will learn to predict.

⚠ ML models cannot handle missing values — always clean your data first.

Encoding Categorical Data & Splitting

Label Encoding

ML works with numbers only. Use `LabelEncoder` to convert text labels:

```
from sklearn.preprocessing import
LabelEncoder
le = LabelEncoder()
df['Placement'] =
le.fit_transform(df['Placement'])
# Yes → 1,  No → 0
```

Train-Test Split

Split data to evaluate model performance on unseen examples:

```
from sklearn.model_selection
import train_test_split
X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.2,
random_state=42
)
```

Training

80%

Testing

20%

Introduction to Scikit-Learn

- Scikit-Learn is the industry-standard Python library for classical machine learning
  — easy to learn, richly documented, and packed with algorithms.

Classification

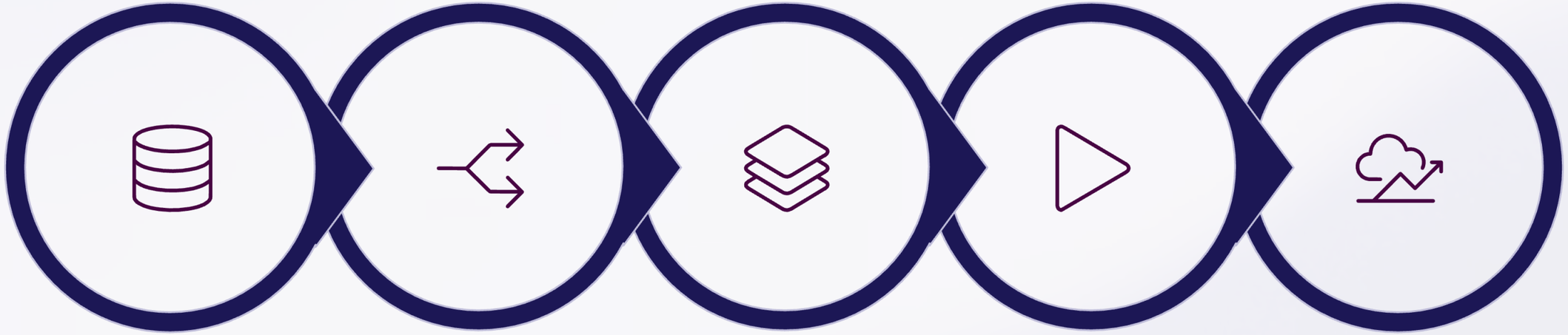
Regression

Clustering

Dimensionality Reduction

Model Evaluation

Scikit-Learn Workflow



Dataset **Train-Test** **Model Select** **Training** **Prediction**

Scikit-Learn enforces a consistent `fit → predict → evaluate` pattern across all algorithms, making it easy to swap models without rewriting your pipeline.

Classification & Predictions

Logistic Regression

```
from sklearn.linear_model
import LogisticRegression

model = LogisticRegression()
model.fit(X_train, y_train)
```

During training, the model learns patterns from labeled data to distinguish between classes.

Making Predictions

```
predictions =
model.predict(X_test)
print(predictions)
```

Evaluating Accuracy

```
from sklearn.metrics import
accuracy_score
accuracy_score(y_test, predictions)
# Output: 0.90
```

90% prediction accuracy on the test set.

Popular Scikit-Learn Algorithms

1	2	3
Classification • Logistic Regression • Decision Tree • Random Forest • SVM • KNN	**Regression** • Linear Regression • Polynomial Regression	**Clustering** • K-Means • DBSCAN

Scikit-Learn's broad algorithm collection covers nearly every classical ML use case, all sharing the same consistent API.

Predicting Student Pass or Fail with Logistic Regression

A step-by-step walkthrough of building a binary classification model using Python and scikit-learn — from raw data to accurate predictions.

Goal

Predict whether a student passes or fails based on the number of hours they studied.

- Input (X): Hours Studied
- Output (y): Pass (1) or Fail (0)

Dataset

Hours Studied	Pass? (1=Yes, 0=No)
1	0
2	0
3	0
4	0
5	1
6	1
7	1
8	1

The 7-Step Workflow

01

Prepare Data

Load and structure study hours and pass/fail labels.

02

Split Data

Divide into training and testing sets.

03

Create Model

Instantiate a Logistic Regression classifier.

04

Train Model

Fit the model on training data.

05

Make Predictions

Run the model on test data.

06

Compare Results

Check predicted vs. actual labels.

07

Calculate Accuracy

Measure how well the model performed.

Step 1: Prepare the Data

- We use NumPy to structure our dataset.
- `X` holds study hours as a 2D array, and
- `y` holds the binary labels — 0 = Fail, 1 = Pass.

```
import numpy as np


X = np.array([[1],[2],[3],[4],[5],[6],[7],[8]])
y = np.array([0, 0, 0, 0, 1, 1, 1, 1])
```

X must be a 2D array (each row is one sample) while y is a 1D array of labels.

Step 2: Split Training & Testing Data

We use `train_test_split` with `test_size=0.25`, reserving 25% of data for testing. The model learns from training data and is evaluated on test data.

```
from sklearn.model_selection import train_test_split


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
```

Training Data (6 samples)

Hours	Result
1	0 (Fail)
2	0 (Fail)
4	0 (Fail)
5	1 (Pass)
7	1 (Pass)
8	1 (Pass)

Testing Data (2 samples)

Hours	Actual
3	0 (Fail)
6	1 (Pass)

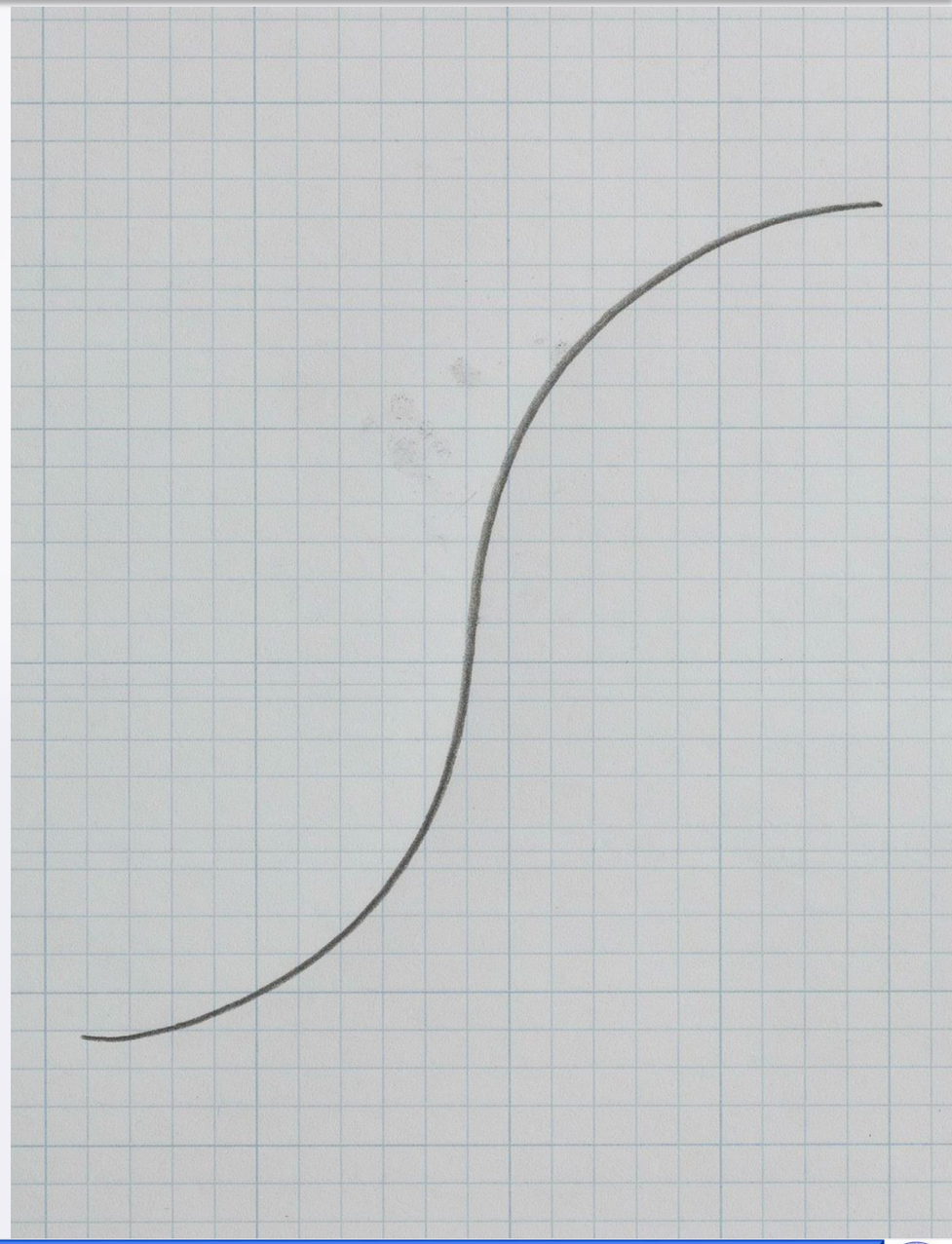
Step 3: The Logistic Regression Model

Logistic Regression estimates the probability of passing using the sigmoid function, which maps any value to a range between 0 and 1.

$$P(y = 1) = \frac{1}{1 + e^{-z}} \quad \text{where} \quad z = b_0 + b_1 x$$

```
from sklearn.linear_model import
LogisticRegression


model = LogisticRegression()
```



Step 4: Train the Model

Calling `model.fit()` teaches the model the relationship between study hours and passing. It discovers the optimal decision boundary that separates Pass from Fail.

```
model.fit(X_train, y_train)
```

Hours < 4.5	**Hours ≥ 4.5**
Fail (0) — Insufficient study time	Pass (1) — Sufficient study time

After training, the model learns this decision boundary from the data automatically.

Step 5: Make Predictions

We call `model.predict(X_test)` on our two test samples — 3 hours and 6 hours — and the model outputs class labels directly.

```
predictions = model.predict(X_test)
# X_test = [[3], [6]]
# Output: [0 1]
```

Hours Studied	Prediction	Meaning
3	0	Fail
6	1	Pass

Step 6: Compare Predicted vs. Actual

Actual Labels (y_test)

```
[0  1]
```

Ground truth from our original dataset.

Predicted Labels

```
[0  1]
```

Output from `model.predict()`.

All predictions match the actual results — the model achieved perfect accuracy on this test set.

Step 7: Calculate Accuracy

The `accuracy_score` function compares predicted and actual labels, returning the fraction of correct predictions.

```
from sklearn.metrics import accuracy_score


accuracy = accuracy_score(y_test, predictions)
print(accuracy)  # Output: 1.0
```

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

For example, with 4 correct out of 5 total predictions: *4/5 = 0.80* — meaning 80% accuracy.

The Complete Program

```
import numpy as np
from sklearn.model_selection import
train_test_split
from sklearn.linear_model import
LogisticRegression
from sklearn.metrics import accuracy_score

# Dataset
X =
np.array([[1],[2],[3],[4],[5],[6],[7],[8]])
y = np.array([0,0,0,0,1,1,1,1])

# Split data
X_train, X_test, y_train, y_test =
train_test_split(
 X, y, test_size=0.25, random_state=42)

# Create, train, predict
model = LogisticRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)

# Evaluate
acc = accuracy_score(y_test, predictions)
print("Predictions:", predictions)
print("Actual:", y_test)
print("Accuracy:", acc)
```

Sample Output

```
Predictions: [0 1]
Actual:      [0 1]
Accuracy:    1.0
```

LogisticRegression()

Creates the classification model.

fit(X_train, y_train)

Trains the model on labeled data.

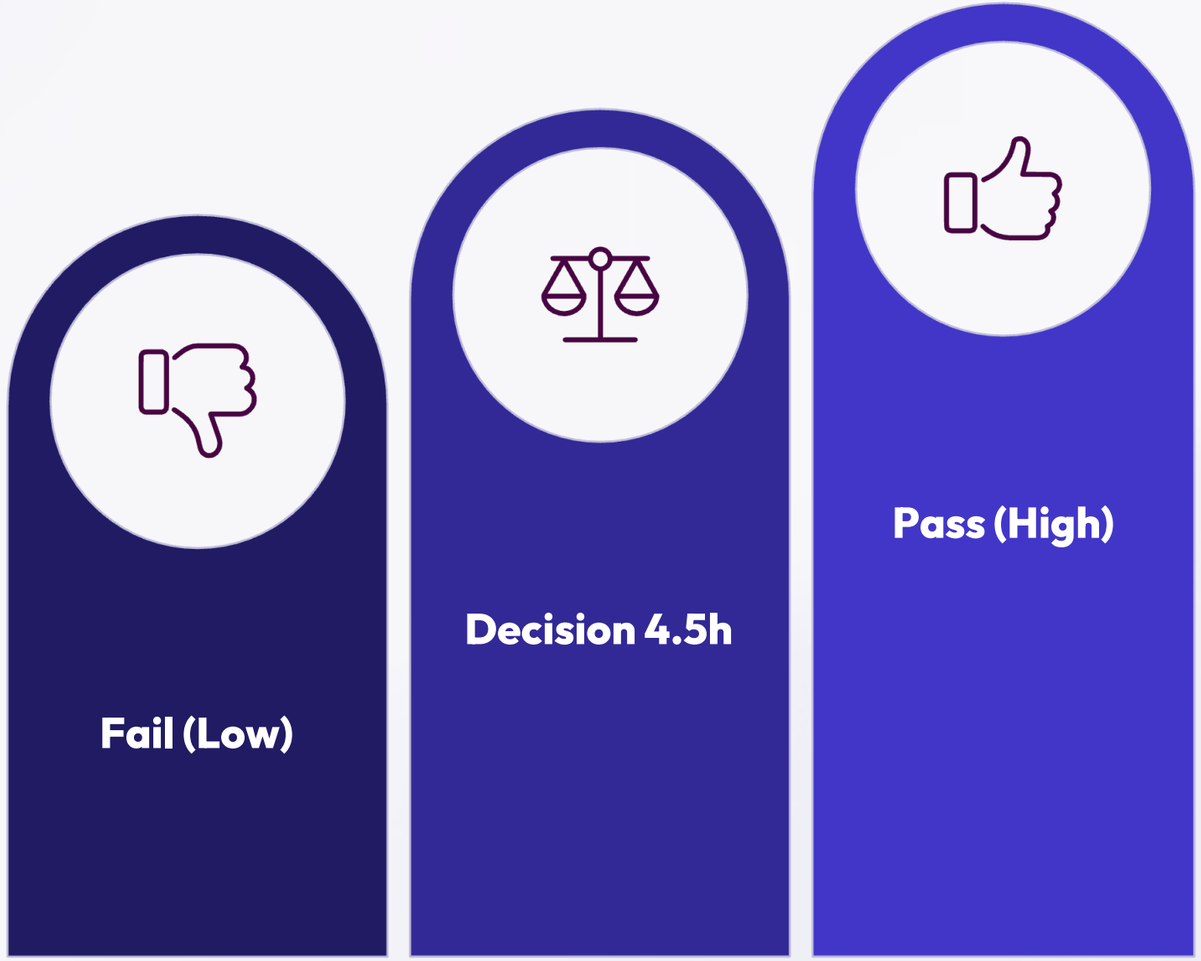
predict(X_test)

Outputs class labels: 0 or 1.

accuracy_score()

Measures prediction correctness.

Understanding the Sigmoid Curve



The sigmoid function smoothly transitions from near-zero probability (Fail) to near-one probability (Pass), with the decision boundary at the midpoint — approximately 4.5 hours of study in our example.

Real-World Applications

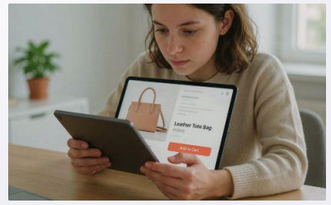
Logistic Regression is one of the most widely used algorithms for **binary classification** — any problem where the answer is one of two outcomes.

Spam Detection

Classify emails as **Spam** or **Not Spam**.

Disease Diagnosis

Predict **Disease** or **No Disease** from patient data.



Customer Purchase

Predict whether a customer will **Buy** or **Not Buy**.

Key Takeaways

Binary Classification

Logistic Regression predicts one of two outcomes — perfect for Pass/Fail, Yes/No problems.

Sigmoid Function

Maps any input to a probability between 0 and 1, enabling confident classification decisions.

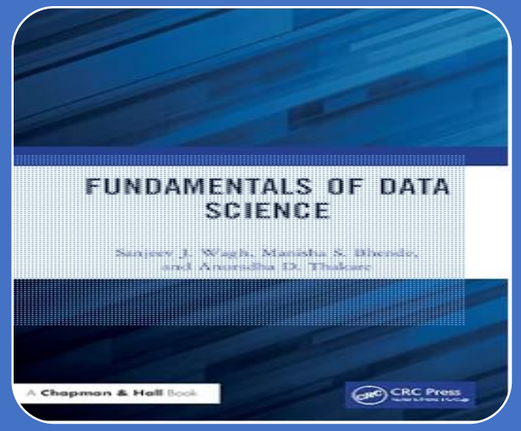
Train → Predict → Evaluate

The three-phase ML workflow: fit on training data, predict on test data, measure with accuracy score.

Widely Applicable

From spam filters to medical diagnosis, Logistic Regression powers real-world classification at scale.

Reference



T1. Sanjeev Wagh, Manisha Bhende, Anuradha Thakare, ‘Fundamentals of Data Science, CRC Press, 1 st Edition, 2022.

P.No.:138-142